

RPKIにおけるPQC移行方式の研究設計

Next Trust Anchorから主要なCA運用主体までの認証経路を事前に構築し、Hosted CAを必要時に生成しながらCA単位でNext Suiteへ移行する方式

既存のTA key staging、CA key rollover、Mixed-TreeのCA単位移行を組み合わせ、Current Suite危殆化後の移行可能性と平常時の運用コストを両立させる。

目次

- | | |
|--------------------|--------------|
| 1. 用語 | 9. RPの処理 |
| 2. 問題設定 | 10. 安全性 |
| 3. 方式の構成 | 11. コストモデル |
| 4. Next TAの事前受理 | 12. 既存方式との比較 |
| 5. 事前構築するCAの範囲 | 13. ステークホルダー |
| 6. Hosted CAの必要時生成 | 14. 新規性 |
| 7. Delegated CAの扱い | 15. 実装・評価 |
| 8. CA単位の移行手順 | 16. 未解決事項 |

1. 用語

Current Suite	現在のRPKIで使用する署名・ハッシュアルゴリズムの組。
Next Suite	移行先として準備する署名・ハッシュアルゴリズムの組。PQCまたは古典方式とPQCの複合構成を想定する。
Hosted CA	Resource holder用の論理CAをRIRまたはNIRが代理運用する構成。CA鍵、証明書、CRL、Manifest、signed objects、publicationはHosted service operatorが管理する。
Delegated CA	Resource holderまたは別の事業者が自らCAを運用する構成。親CAとはProvisioning Protocolで連携し、自ら鍵、CRL、Manifestおよびpublicationを管理する。
事前構築されたNext path	Current Suiteが安全な期間に構築され、事前受理されたNext Trust Anchorから Next Suiteだけで検証できるCA証明書path。
CA単位の移行	兄弟CAの進捗とは独立に、新しいCA instanceをstageし、signed productsを再発行してNext Suiteへ切り替えること。

2. 問題設定

Current Suiteが偽造可能になった後、Current署名だけを用いて新しいNext公開鍵を認証すると、攻撃者が自らのNext公開鍵を正規CAの鍵として導入できる。Next署名方式自体が安全でも、公開鍵の正当性を破られたCurrent pathだけで決定する構成は安全ではない。

Current親CA

- └ Current署名でNext子CA鍵を認証
 - └ Next署名のROA

Current署名を偽造できる場合:

攻撃者は任意のNext子CA鍵を含む証明書を作成できる

一方、Next Trust Anchorから全論理CAまでを事前に構築すると、CA証明書、鍵、CRL、Manifestの数が現在のCA数に比例して増加し、移行開始前から継続的な更新負荷が発生する。

本方式では、Current Suite破壊後にも新しいNext CA証明書を安全に発行できるCA運用主体までNext pathを事前構築し、その配下の論理CAは必要時に生成する。

3. 方式の構成

Trust Anchorの準備

RFC 9691のsuccessor key通知、相互確認、acceptance timer、ローカル状態管理を基に、Next Trust AnchorをCurrent Suite危殆化前にRPへ受理させる。

上位CA pathの準備

TA、RIR、NIR、および独立運用される主要なDelegated CAまでをNext Suiteで事前構築する。

CA単位の移行

Hosted CAやその配下の論理CAは必要時に生成し、RFC 6489のstagingとMixed-TreeのCA単位移行を用いて順次切り替える。

Current Suite

Current TA

- └ Current RIR/NIR
 - └ Hosted CA A
 - └ Hosted CA B
- └ Delegated ISP CA
 - └ Customer X
 - └ Customer Y

事前生成しない例:

- Hosted CA A / B
- Hosted operatorが必要時に生成できるleaf CA
- Next Delegated ISP CAが必要時に生成できる配下CA

Next Suite (危殆化前)

Next TA

- └ Next RIR/NIR
 - └ Next Delegated ISP CA

Mixed-Treeから利用するのは、CAごとの非同期移行とRFC 6489型のstagingである。Current Suite危殆化後に、新しいNext鍵をCurrent署名で認証するmixed certificateには依存しない。

4. Next Trust Anchorの事前受理

Next Trust Anchorの公開鍵は、Current Suiteが安全な期間にRPへ伝え、受理済み状態として保存する必要がある。RFC 9691はTAK objectによるsuccessor public keyと証明書URIの通知、predecessorとの相互確認、acceptance timer、およびローカル状態管理を定義している [1]。

RFC 9691では、timer満了後にsuccessor keyがCurrent keyを置き換え、RPは通常一つのTA keyをproduction validationに使用する。本方式では、Current TAを使用している期間にもNext TAを並行するNext pathの検証に使用する。したがって、TAKの考え方を利用するが、acceptance後のRP処理はRFC 9691のTA key rolloverと異なる。

既存TAKの意味を変更して利用するか、別のprofileまたはobjectで用途を区別するかは標準化上の検討事項である。

5. 事前構築するCAの範囲

事前構築の単位は、論理CA数ではなく、Current Suite破壊後にもNext Suiteで子CA証明書を発行できる独立したCA運用主体を基準に決める。

CAの種類	事前準備	理由
Trust Anchor	必須	すべてのNext pathの信頼起点になる。
RIR / NIR	必須	Hosted CAおよびDelegated CAへNext Suiteの証明書を発行する。
Hosted resource holder CA	原則として必要時生成	親CAと子CAを同じHosted backendが管理し、資源情報とROA設定を保持している。
Delegated CA	原則として事前構築	親子が異なるoperatorであり、危殆化後のprovisioningと組織認証への依存を避ける。
子CAを持つ Delegated CA	事前構築を優先	未対応の場合、その配下のsubtree全体がNext pathへ接続できない。

RFC 9691は複数のTA key pairを維持する場合、各key pairの下で同等のresource delegationを維持し、変更を両方へ反映することを要求している [1]。本方式では同じ原則を、事前構築する上位CA間のCurrent/Next resource stateへ適用する。

6. Hosted CAの必要時生成

Hosted RPKIでは、RIR/NIRが各resource holderの論理CAを代理運用し、秘密鍵生成、証明書発行、signed object生成、publicationを行う。APNICとRIPE NCCのHosted serviceもこの運用モデルを採用している [6][8]。

Next Trust AnchorからHosted service operatorのNext CAまでのpathが事前構築されていれば、個別のHosted CAはCurrent Suite危殆化後に生成できる。

危殆化前

Next TA

└ Next RIR/NIR CA

危殆化後または計画移行時

Next TA

└ Next RIR/NIR CA

└ Next Hosted CA A

└ CRL

└ Manifest

└ ROA

└ ASPA

1 Hosted backendがNext CA鍵を生成する

利用者がEE鍵やCA鍵を操作する必要はない。Next Suiteに対応したHosted backendが鍵を生成・保管する。

2 Next RIR/NIR CAが資源証明書を発行する

RIR/NIRの権威的な資源登録情報を基に、Next Hosted CAへIP address resourcesとAS resourcesを委任する。証明書はNext Suiteで署名されるため、Current署名を使用しない。

3 CRL、Manifest、signed objectsを生成する

Hosted backendに保存されているROA・ASPA設定を用いてNext側のproductsを生成し、Next repositoryへ公開する。

RPはNext TAからNext RIR/NIR CA、Next Hosted CA、EE certificate、signed objectまでを Next Suiteだけで検証する。

この手順が安全に成立する条件は、Next RIR/NIR pathが事前構築済みであること、Hosted backendの資源データと利用者認証が正しいこと、Next Suiteの鍵管理・発行機能が利用可能であること、およびRPがCurrent側へfallbackしないことである。

Hosted CA自身のNext鍵と証明書を事前に維持しなくても、Next親CAとHosted operatorの管理系が安全であれば、Current Suite危殆化後にNext pathを生成できる。

7. Delegated CAの扱い

Delegated RPKIでは、親CAと子CAが異なるoperatorによって管理される。RFC 8183はProvisioning ProtocolおよびPublication Protocolの関係を設定するための out-of-band public key exchangeを定義している [4]。

Current Suite危殆化後に初めてNext公開鍵を登録する場合、親CAはCurrent RPKI署名以外の方法でoperatorとNext公開鍵の正当性を確認しなければならない。この依存を避けるため、初期設計ではDelegated CAをNext Suiteで事前構築する。

危殆化前に維持

Next TA

- └ Next RIR/NIR CA
 - └ Next Delegated CA
 - └ CRL
 - └ Manifest

移行時または危殆化後

Next Delegated CA

- └ ROA / ASPA
- └ 必要に応じてNext child CA certificates

Repository負荷をさらに減らす案として、Next公開鍵またはSPKI hashだけを危殆化前に登録し、証明書とRepository objectsを必要時に生成する方式も考えられる。ただし、登録情報の更新、proof of possession、resource変更、通常のkey rollover、危殆化後の変更禁止を追加で定義する必要があるため、主要方式とは分けて評価する。

8. CA単位の移行手順

各CAの切替には、RFC 6489のCURRENT、NEW、OLD CA instanceとStaging Periodの考え方を利用する。RFC 6489ではNEW CAが最初に空のCRLとCRLだけを掲載したManifestを公開し、staging後にCURRENT CAのproductsを再発行して置換する [2]。

1 Next CA instanceを準備する

事前構築済みCAは既存instanceを使用する。Hosted CAはNext親CAの下で必要時に生成する。

2 CRLとManifestをstageする

Next CAが有効であり、RPから取得・検証できる状態を確認する。

3 signed productsを再発行する

Current側と同じ意味を持つROA、ASPA、子CA証明書等をNext Suiteで生成する。比較は正規化したVRP set、ASPA payload set、resource set等に対して行う。

4 対象CAをNext側へ切り替える

兄弟CAの移行を待たずに、対象CA配下の検証結果をNext側へ切り替える。切替後のNext対応RPはCurrent側へfallbackしない。

5 Current CA instanceを廃止する

Legacy RP向けの猶予期間または共通EOLの終了後、Current証明書とproductsを失効・撤去する。

あるCAがNextへ移行しても、兄弟CAはCurrentのまま運用できる。この意味で一つのRPKI全体にCurrentとNextのCAが混在する。ただし、危殆化後に生成するNext CA証明書は事前構築済みのNext親CAから発行し、Current親CAの署名には依存しない。

9. RPの処理

状態	Next path	Next products	RPの処理
Currentのみ	未受理または未構築	なし	Current側を使用する。
上位path準備済み	TAからCA operatorまで有効	対象Hosted CAは未生成でもよい	Current側を使用し、Next上位pathを継続検証する。
Staging	対象CAまで有効	並行発行	Current側を出力し、Current/Nextの意味的同等性を確認する。
切替後	対象CAまで有効	正式	対象CA配下ではNext側を使用し、Currentへfallbackしない。
Current廃止後	有効	正式	Next側のみを使用する。

Current側とNext側の比較は、CCRを必須要素とせず、CA証明書から得たresource set、Validated ROA Payload、Validated ASPA Payload等を直接正規化して比較する。

CA単位の切替をRPへ通知する方法は別途定義する必要がある。必要な情報は、対象CAまたはresource class、対応するCurrent/Next CA、切替状態、更新順序、およびrollbackを防ぐための永続状態である。

10. 安全性

条件または攻撃	処理
Current署名の偽造	危殆化後にCurrent署名で導入された新しいNext CA鍵を信頼しない。
Hosted CAの新規生成	事前構築済みNext RIR/NIR CAが、権威的な資源登録情報を基にNext Suiteで証明書を発行する。
Delegated CAの移行	事前構築済みNext CAを使用する。未準備の場合は安全なout-of-band provisioningが追加で必要になる。
Next repositoryの妨害	切替後はCurrentへfallbackせず、可用性障害として扱う。
古いCurrent状態のReplay	RPが切替済み状態と更新順序を永続化し、Currentへ戻らない。
Hosted管理系の侵害	本方式では防止しない。利用者認証、資源登録DB、HSM、CA backendを別途保護する必要がある。

Hosted CAが危殆化後に安全に移行できるのは、Current RPKI署名ではなく、事前構築済みのNext親CAとRIR/NIRの権威的な資源登録情報を信頼するためである。

11. コストモデル

全論理CA数を N 、事前構築する独立CA運用主体数を B 、同時に移行中のCA数を M とする。

方式	平常時に維持するNext CA	移行時の追加 objects	危殆化後の移行
全CAを事前構築	N 個	M 個分のROA・ASPA等	全CAが直ちに移行可能
Mixed-Treeのみ	ほぼなし	M 個分のCAと products	Current署名に依存する未準備CAは安全な鍵導入が困難
本方式	B 個	M 個分のCAと products	Hosted CAはNext親CAから必要時生成。 Delegated CAは事前準備範囲まで移行可能

全CA事前構築:

$N \times (\text{Next CA key} + \text{CA certificate} + \text{CRL} + \text{Manifest})$

本方式:

$B \times (\text{Next CA key} + \text{CA certificate} + \text{CRL} + \text{Manifest})$

+

$M \times (\text{移行対象CAのNext products})$

通常は $B \ll N$ となることを期待するが、
世界全体の比率は実測が必要である。

Hosted CAを事前生成しないことで、待機期間中のCA鍵、CRL、Manifest、Manifest用EE鍵、Repository容量、RRDP更新を削減できる。一方、危殆化時に大量のHosted CAを一斉生成すると、鍵生成、証明書発行、publication、RP初回検証が集中するため、batch生成と定期訓練が必要になる。

12. 既存方式との比較

項目	RFC 6916	Mixed-Tree	本方式
Next TA	並行SuiteのTALを使用	移行設計に応じて準備	RFC 9691の考え方を基に危殆化前に受理
事前構築範囲	top-downに広範な並行階層を構築	移行対象CAを必要時に準備	TA、RIR/NIR、主要Delegated CAまでを準備
Hosted CA	並行階層の一部として事前発行	CA単位で移行	Next親CAから必要時に生成
ROA·ASPA等	移行phaseに従い両Suiteで発行	対象CAの切替時に再発行・置換	対象CAだけ再発行し、兄弟CAと独立に切替
危殆化後の新しいNext鍵	完成済みNext hierarchyを使用	Current署名だけに依存すると安全ではない	事前構築済みNext親CAからNext Suiteで証明書を発行
平常時のRepository負荷	大	小	中。独立CA運用主体数に比例

RFC 6916は計画的かつtop-downな移行を定義し、emergency transitionは対象外としている [3]。本方式は、危殆化前のNext trust stateを利用して、Current Suite破壊後にもHosted CAを生成できる条件を追加する。

13. 実装・展開ステークホルダー

論理CA数と、独立してCA softwareを実装・運用する組織数は一致しない。Hosted RPKIでは一つのRIR/NIR backendが多数の論理CAを運用する。APNICはHosted Modeを多くのMemberに推奨し [6]、RIPE NCCもHosted serviceを多くの利用者に適した方式と説明している [7]。ARINは2026年第1四半期にRPKI参加組織の98%がHosted RPKIを利用していたと報告している [9]。

集中して対応する主体

- RIR/NIRのTA・Hosted CA backend operator
- RPKI CA software実装者
- RP software実装者
- 主要なDelegated CA operator

一般のHosted利用者

- CA鍵やEE鍵を直接管理しない
- Next Suite実装を自組織へ導入しない
- ROA・ASPA設定と切替方針を確認する

Hosted RPKIの普及は、独立したsoftware deployment、operator coordination、個別HSM、publication運用の数を削減する。RP検証量と、実際に移行したHosted CAの証明書・CRL-Manifest数は削減しない。

14. 新規性

1. Next pathの事前構築範囲を全論理CAではなく、Current Suite破壊後に安全な子CA発行を継続できるCA運用主体までに限定する。
 2. Hosted CAについて、事前構築済みNext親CAと権威的な資源登録情報を用い、Current Suite危殆化後にNext CAを必要時生成する。
 3. Delegated CAには事前構築を要求し、Hosted CAには必要時生成を許すことで、trust boundaryに応じて準備コストを変える。
 4. RFC 6489のstagingとMixed-TreeのCA単位移行を、Current署名に依存しないNext-rooted pathの下で使用する。
 5. 事前構築CA数、Repository負荷、危殆化後に移行可能なresource割合の関係を定量化する。
-

15. 実装・評価

実装段階

1. Current TAとNext TAを用意し、RFC 9691を基にしたNext TA事前受理をprototype化する。
2. Next TAからRIR/NIRおよびDelegated CAまでのNext pathを構築する。
3. Hosted CAを事前生成しない状態で、Current側のROA設定と資源情報を保持する。
4. Current Suiteを無効とした後、Next RIR/NIR CAからHosted CAを新規生成する。
5. Next CRL、Manifest、ROA、ASPAを公開し、Next TAから検証する。
6. RFC 6489型staging、意味的比較、CA単位切替、Current fallback禁止を実装する。
7. Next SuiteをP-256からML-DSAまたは複合署名へ置き換える。

比較する構成

- RFC 6916型の全面並行階層
- 全論理CAまでNext pathを事前構築
- 本方式: RIR/NIR・Delegated CAまで事前構築し、Hosted CAは必要時生成
- Next公開鍵だけを事前登録するDelegated CA最適化
- Mixed-Treeのみ

測定項目

分類	測定内容
Repository	CA証明書、CRL、Manifest、ROA、ASPAの数と容量、RRDP Snapshot/Delta
CA	平常時の定期更新、Hosted CA一括生成時間、証明書発行、publication時間
RP	複数TA validation、署名検証数、増分検証、切替状態の保存
安全性	Current署名偽造、偽Next鍵、Repository Replay、Next妨害、fallback
運用	独立CA operator数、Hosted/Delegated比率、HSM鍵数、障害時batch規模
移行能力	危殆化後に安全に移行可能なCA数、resource量、subtree数

16. 未解決事項

- RFC 9691のTAKを既存形式のまま利用するか、別用途を示すprofileまたはobjectを定義するか。
 - CA単位の切替状態とrollback防止情報をRPへ伝える標準方式。
 - CurrentとNextのCA証明書およびvalidated payloadの意味的同等性を比較する共通手順。
 - Delegated CAのNext公開鍵だけを事前登録する場合のproof of possessionと更新規則。
 - 通常のCA key rollover、resource変更、Suite migrationが同時発生する場合の処理順序。
 - 危殆化時に多数のHosted CAを生成・公開するためのbatch制御とRepository負荷。
 - Hosted管理系、会員認証、資源登録DBがCurrent Suiteとは独立して安全であることの運用要件。
-

17. 参考資料

1. RFC 9691 — A Profile for Resource Public Key Infrastructure Trust Anchor Keys
 2. RFC 6489 — Certification Authority Key Rollover in the RPKI
 3. RFC 6916 — Algorithm Agility Procedure for the RPKI
 4. RFC 8183 — Out-of-Band Setup Protocol for RPKI Production Services
 5. D. Doesburg — Post-Quantum Cryptography for the RPKI
 6. APNIC — RPKI deployment models
 7. RIPE NCC — Using a Delegated Certification Authority
 8. RIPE NCC — Hosted and Delegated RPKI
 9. ARIN 57 Day 2 Recap — RPKI adoption and Hosted RPKI usage
-